GrokRxiv:2026.03.tool-interface-ai-os [cs.AI] 10 Mar 2026

The AI Operating System, Part II: Tool Interface Layer — Composition, Capabilities, and the Elixir/OTP Reference Implementation

Matthew Long
The YonedaAI Collaboration
YonedaAI Research Collective
Chicago, IL
matthew@yonedaai.com · https://yonedaai.com

March 10, 2026

Abstract

An AI Operating System mediates every interaction between intelligent agents and the external world through a *tool interface layer* that is analogous to the device-driver subsystem of a classical operating system. In this second paper of a five-part series, we design a tool interface layer built on the principles of functional composition, capability-based security, and sandboxed execution. The design centres on a three-tier tool registry—Builtin, Sandbox, and MCP (Model Context Protocol)—where tools are higher-order functions composed through pipelines, access is governed by HMAC-signed capability tokens with time-to-live and rate limiting, and untrusted code executes in isolated BEAM processes with hard timeouts. We present a complete reference implementation in Elixir/OTP comprising six modules: a GenServer-based registry with freeze semantics, a cryptographic capability token system, a process-isolated sandbox executor, a JSON-RPC MCP protocol client with parallel discovery, an asynchronous audit logger with telemetry integration, and a facade module that composes these components into a single invocation pipeline. The implementation exploits the BEAM virtual machine's natural properties—per-process heap isolation, preemptive scheduling, supervision trees, and message-passing concurrency—to achieve the security and reliability guarantees that classical operating systems obtain through hardware-enforced protection rings. We validate the design against the Agent-Hero production platform, where the three-tier model maps directly to builtin tools, E2B ephemeral sandboxes, and `StreamableHTTPClientTransport`-based MCP connections. Comparisons are drawn to Unix device drivers, Plan 9, Capsicum capabilities, and seL4 verified security.

Keywords: AI operating system, tool interface, capability-based security, Model Context Protocol, functional composition, Elixir/OTP, sandboxing, pipeline operators, higher-order functions

Contents

1 Introduction **4**

2 Background and Motivation **5**
2.1 Device Drivers and the Abstraction Principle . . . 5
2.2 Capability-Based Security . . . 5
2.3 The BEAM Virtual Machine . . . 6
2.4 Functional Programming as Design Language . . . 6

3 Three-Tier Tool Registry Design **7**
3.1 Design Overview . . . 7
3.2 Tool Specification as a Record Type . . . 8
3.3 The Registry GenServer . . . 9
3.4 Lookup: Searching Across Tiers . . . 9
3.5 MCP Registration with Namespacing . . . 10
3.6 Freeze Semantics: Contract-Locked Configuration . . . 10
3.7 Builtin Tool Definitions . . . 11
3.8 URL Safety Validation . . . 12
3.9 Listing and Filtering . . . 12

4 Capability Token System **13**
4.1 Design Philosophy . . . 13
4.2 Token Structure . . . 14
4.3 Token Creation with HMAC Signing . . . 14
4.4 Authorization: The Five-Check Pipeline . . . 16
4.5 Rate Limiting with Sliding Windows . . . 17
4.6 Cooperative Revocation . . . 17

5 Sandboxed Execution **18**
5.1 Isolation Through Process Boundaries . . . 18
5.2 The Execute Function . . . 18
5.3 Parallel Sandbox Execution . . . 20
5.4 URL Validation for Sandbox Tools . . . 20

6 MCP Protocol Client **21**
6.1 The Model Context Protocol . . . 21
6.2 Session Initialization . . . 21
6.3 Tool Discovery and Registration . . . 22
6.4 Closures as Remote Executors . . . 23
6.5 Remote Tool Invocation . . . 23
6.6 Parallel Multi-Server Discovery . . . 24
6.7 Authentication Headers . . . 25

7 The Invocation Pipeline: Facade Module **25**
7.1 Composing the Components . . . 25
7.2 Tier-Aware Execution Dispatch . . . 26
7.3 Input Validation . . . 27
7.4 Public API Functions . . . 27
7.5 OTP Application and Supervision . . . 28

8 Audit Logger **29**
8.1 Design: Asynchronous, Non-Blocking Logging . . . 29
8.2 Entry Structure . . . 29
8.3 Asynchronous Logging with Telemetry . . . 29
8.4 Sensitive Data Sanitization . . . 30
8.5 Ring Buffer with Capacity Bounds . . . 30
8.6 Periodic Flushing . . . 31
8.7 Query and Statistics . . . 32

9 Production Validation: Mapping to Agent-Hero **32**
9.1 Three-Tier Model Correspondence . . . 33
9.2 Capability Token Correspondence . . . 33
9.3 Sandbox Correspondence . . . 33
9.4 MCP Correspondence . . . 34
9.5 Audit Correspondence . . . 34

10 Design Analysis and Comparisons **35**
10.1 Comparison to Classical Operating System Patterns . . . 35
10.2 Comparison to Plan 9 . . . 35
10.3 Comparison to Capsicum and seL4 . . . 35
10.4 Functional Composition Properties . . . 36
10.5 Security Properties . . . 36
10.6 Scalability Properties . . . 36

11 Case Studies **37**
11.1 Web Scraping Pipeline . . . 37
11.2 Sandboxed Code Execution . . . 37
11.3 Multi-MCP Server Discovery . . . 38

12 Security Limitations and Future Directions **39**
12.1 Known Limitations . . . 39
12.2 Future Directions . . . 39

13 Related Work **40**
13.1 Tool Use in Language Models . . . 40
13.2 Capability-Based Security . . . 40
13.3 Actor Model and BEAM . . . 40
13.4 Operating System Abstractions . . . 41

14 Conclusion **41**

1 Introduction

The fundamental architectural insight of classical operating systems is that application code must never interact with hardware directly. Instead, a layer of *device drivers* interposes between user-space programs and peripheral devices, providing abstraction, multiplexing, isolation, and auditability (Tanenbaum & Bos, 2014). An application opens a file descriptor, reads and writes through a well-defined system-call interface, and the kernel routes those operations to the appropriate driver. The application need not know whether the underlying device is a local disk, a network socket, or a USB peripheral.

In an AI Operating System, the same architectural principle applies with even greater force. Agents—large language models with autonomy over their computational environment—must interact with external systems: databases, web APIs, file systems, code interpreters, and other agents. These interactions are mediated by *tools*: typed functions that accept structured input and return structured output, with well-defined security boundaries, execution constraints, and audit trails.

This paper develops the design and implementation of the tool interface layer. Our approach is rooted in functional programming: tools are composable functions, security is enforced through unforgeable capability tokens, isolation is achieved through process boundaries, and the entire invocation pipeline is a composition of pure validation, authorized execution, and audit logging. While the underlying ideas can be formalized using category theory (tools as morphisms, capabilities as representable functors), we focus on the engineering design and its concrete realization in Elixir/OTP.

Contributions. Our contributions are:

1. A **three-tier tool registry design** (Builtin, Sandbox, MCP) with freeze semantics that locks tool configurations at execution time, preventing privilege escalation through dynamic discovery (Section 3).

2. A **capability-based security model** using HMAC-signed tokens with time-to-live, rate limiting, and permission scoping, enforcing the Principle of Least Authority without centralized access control lists (Section 4).

3. A **sandboxed execution model** using BEAM process isolation with monitored timeouts, providing the same isolation guarantees as E2B ephemeral environments but at the language-runtime level (Section 5).

4. A **complete Elixir/OTP reference implementation** comprising six modules totaling approximately 700 lines, with GenServer state machines, supervision trees, and telemetry integration (Sections 7–8).

5. A **MCP protocol client** with session management, parallel multi-server discovery via `Task.async_stream`, and closure-based remote execution (Section 6).

6. **Production validation** against the Agent-Hero platform, demonstrating that the reference implementation's design maps directly to a deployed system handling real agent workloads (Section 9).

Relation to the series. This is Part II of a five-part series on the AI Operating System. Part I (Long, 2026a) established the kernel architecture—the process model and scheduling layer. The present paper builds on that foundation by defining the interface through which scheduled agent processes access external capabilities. Part III will address memory and persistence, Part IV the agent orchestration layer, and Part V the integration of all components.

Notation. We use Elixir syntax throughout. The pipe operator `|>` denotes left-to-right function composition: `x |> f |> g` means "apply `f` to `x`, then apply `g` to the result." Pattern matching on tagged tuples (`{:ok, value}` and `{:error, reason}`) provides exhaustive error handling. The `with` expression chains multiple pattern-matched operations, short-circuiting on the first failure.

2 Background and Motivation

2.1 Device Drivers and the Abstraction Principle

The device driver model in Unix-like operating systems provides three critical properties (Love, 2010):

1. **Abstraction.** Applications interact with a uniform file-descriptor interface regardless of the underlying device.
2. **Isolation.** A misbehaving driver cannot corrupt unrelated kernel state, especially in microkernel designs.
3. **Multiplexing.** Multiple processes can share a single device through the driver's queuing and scheduling logic.

Plan 9 from Bell Labs extended this model to its logical conclusion: every resource—networks, graphics, even per-process name spaces—is presented as a file system (Pike et al., 1995). This "everything is a file" philosophy provides a universal interface that any program can consume without special-purpose client libraries. The AI OS tool layer aspires to the same universality: every external capability, whether a web search engine, a code interpreter, or a third-party API, is accessed through the same typed invocation protocol.

2.2 Capability-Based Security

Traditional access control uses access control lists (ACLs) attached to resources. Capability-based security inverts this: each subject holds unforgeable *capability tokens* that grant specific permissions on specific objects (Dennis & Van Horn, 1966; Levy, 1984). Modern realizations include:

- **Capsicum** (FreeBSD): capability mode restricts a sandboxed process to only the file descriptors it was given at entry (Watson et al., 2010).
- **seL4**: a formally verified microkernel where every system call requires a capability argument (Klein et al., 2009).

- **WASI** (WebAssembly System Interface): capability-based access to host resources from sandboxed WebAssembly modules (WASI Working Group, 2023).
- **Macaroons**: decentralized capability tokens with contextual caveats for distributed systems (Birgisson et al., 2014).

The AI OS adapts capability-based security to the agent setting: each agent holds a set of capability tokens that determine which tools it may invoke, with what parameters, and at what rate. Tokens are cryptographically signed, time-bounded, and rate-limited—unforgeable values that travel with the agent rather than being stored in a central authority.

2.3 The BEAM Virtual Machine

The BEAM virtual machine (Erlang/Elixir runtime) provides properties that make it an ideal substrate for a tool interface layer (Armstrong, 2003; Juric, 2019):

- **Process isolation**: Each process has its own heap. There is no shared mutable state between processes.
- **Message passing**: Processes communicate only through asynchronous, copied messages.
- **Preemptive scheduling**: The BEAM scheduler preempts processes after a reduction count, preventing any single tool execution from starving others.
- **Supervision trees**: Processes are organized hierarchically; a supervisor restarts failed children according to a declared strategy.
- **Hot code loading**: Modules can be upgraded at runtime without stopping the system.

These properties mean that BEAM process isolation provides the same security boundary that classical operating systems achieve through hardware protection rings. A sandboxed tool execution in a spawned BEAM process cannot read the parent's memory, cannot access file descriptors it was not given, and is automatically cleaned up if it crashes or times out.

2.4 Functional Programming as Design Language

We frame the tool interface layer using the vocabulary of functional programming rather than, say, object-oriented design patterns or formal category theory. The key concepts are:

- **Composition**: Tools are functions that compose. The output of one tool feeds into the input of the next. Composition is associative: `x |> f |> g |> h` produces the same result regardless of how we parenthesize the intermediate steps.
- **Higher-order functions**: Tools are values. A tool's `execute` field holds a function (`fn input -> {:ok, result} end`). This function can be stored, passed, composed, and wrapped.

- **Closures**: The MCP client constructs remote execution functions as closures that capture the server URL, authentication token, and session ID. The caller invokes the closure without knowing it dispatches over HTTP.
- **Pattern matching**: Authorization, validation, and dispatch are implemented as pattern matches on tagged tuples, providing exhaustive case analysis and clear error paths.
- **Pipeline operators**: The invocation pipeline—authorize, validate, execute, audit—is a left-to-right composition expressed with `with` and `|>`.

Remark 2.1 (Categorical background)**.** For readers familiar with category theory, the design has a clean categorical interpretation: tools are morphisms in a typed category, capability tokens correspond to the representable functor $\mathrm{Hom}(T, -)$, and sandboxing is a passage to a restricted subcategory. The Yoneda lemma guarantees that capability tokens faithfully represent tools. We do not develop this formalism here, but note that the functional programming concepts above are the computational counterparts of these categorical structures.

3 Three-Tier Tool Registry Design

3.1 Design Overview

The tool interface layer organizes tools into three tiers based on trust level and execution model. Each tier represents a different trade-off between flexibility and security:

Design Principle 3.1 (Tiered Trust)**.** Tools are partitioned into three tiers with decreasing trust and increasing isolation requirements:

1. **Builtin** — Statically defined, trusted operations that execute in the registry process itself.
2. **Sandbox** — Operations requiring isolated execution in a separate BEAM process with hard timeouts.
3. **MCP** — Runtime-discovered tools from external Model Context Protocol servers, accessed over HTTP.

The three tiers are not an arbitrary partition. They correspond to three fundamentally different trust relationships:

- **Builtin tools** are authored by the platform developers, reviewed, tested, and deployed as part of the system. They run in the same trust domain as the registry. Examples: `web-search`, `text-transform`, `web-scrape`, `pdf-parse`, `generate-document`, `spreadsheet-read`.
- **Sandbox tools** execute code that may come from untrusted sources (agent-generated code, third-party scripts). They must be isolated: separate heap, hard timeout, monitored lifecycle. Examples: `git-clone`, `npm-run`, `code-exec`, `file-ops`, `python-exec`, `shell-exec`.

- **MCP tools** are discovered at runtime from external servers. The platform has no control over their implementation. They are accessed via HTTP with Bearer token authentication and namespaced to prevent collisions: `${serverName}__${toolName}`.

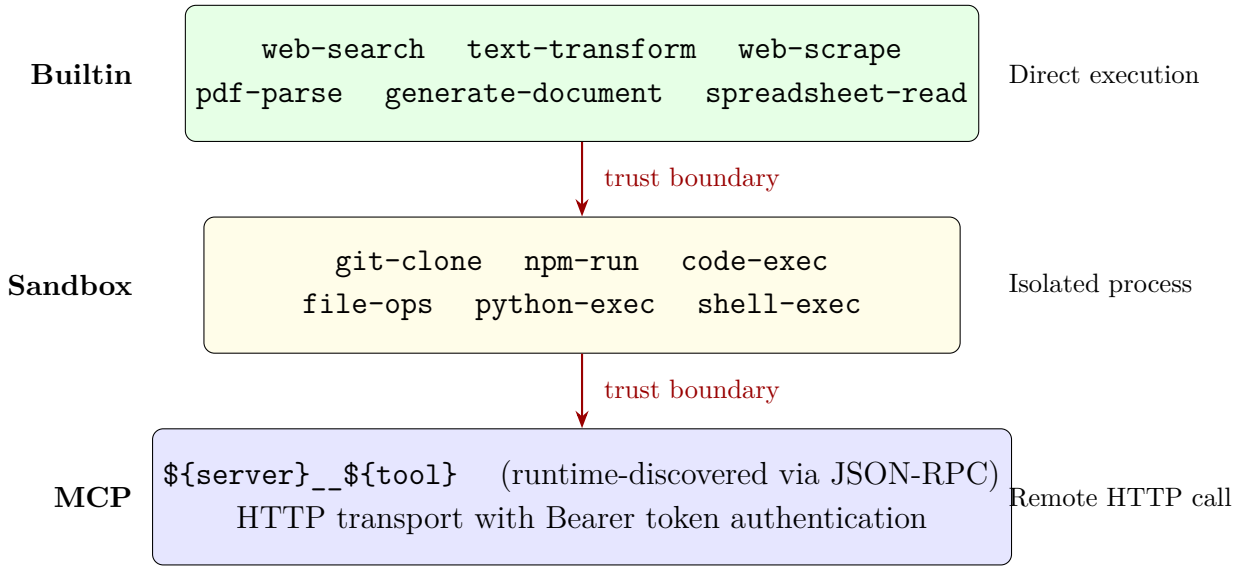

Figure 1: Three-tier tool registry. Trust decreases from top to bottom; isolation increases correspondingly. Each tier boundary represents a different execution model.

3.2 Tool Specification as a Record Type

Every tool, regardless of tier, conforms to a common specification. In Elixir, this is expressed as a typed struct:

```
@type tool_tier :: :builtin | :sandbox | :mcp

@type tool_spec :: %{
  name: String.t(),
  tier: tool_tier(),
  description: String.t(),
  input_schema: map(),            # JSON Schema for input validation
  output_schema: map() | nil,     # JSON Schema for output (optional)
  validate: (map() -> :ok | {:error, term()}) | nil,
  execute: (map() -> {:ok, any()} | {:error, term()})
}
```

Listing 1: Tool specification type.

The critical field is `execute`: a higher-order function that takes a map of input parameters and returns a tagged result tuple. This is the tool's runtime behavior, captured as a first-class value. For builtin tools, the function runs directly. For sandbox tools, it runs inside an isolated process. For MCP tools, it is a closure that dispatches a JSON-RPC call over HTTP.

The `validate` field is an optional higher-order function for input validation. When present, it is called before execution. When absent (nil), all inputs are accepted. This allows tools to define custom validation logic beyond what JSON Schema can express—for example, URL safety checks that block localhost and private IP ranges.

3.3 The Registry GenServer

The registry is implemented as a `GenServer`—an OTP behavior that provides a single-threaded, message-driven state machine. The state holds three maps (one per tier) plus a boolean frozen flag:

```elixir
defmodule ToolInterface.Registry do
  use GenServer
  require Logger

  defstruct builtin: %{}, sandbox: %{}, mcp: %{}, frozen: false

  def start_link(opts \\ []) do
    GenServer.start_link(__MODULE__, opts, name: __MODULE__)
  end

  @impl true
  def init(_opts) do
    state = %__MODULE__{
      builtin: load_builtin_tools(),
      sandbox: load_sandbox_tools(),
      mcp: %{},
      frozen: false
    }
    Logger.info(
      "[Registry] Initialized with #{map_size(state.builtin)} builtin, " <>
      "#{map_size(state.sandbox)} sandbox tools"
    )
    {:ok, state}
  end
end
```

Listing 2: Registry state and initialization.

Builtin and sandbox tools are loaded at startup from static definitions. MCP tools start empty and are populated through runtime discovery. The GenServer's single-threaded message processing guarantees that all registry operations—lookup, registration, freezing—are serialized, eliminating race conditions without explicit locks.

3.4 Lookup: Searching Across Tiers

Tool lookup searches across all three tiers in priority order (builtin first, then sandbox, then MCP):

```elixir
def handle_call({:lookup, tool_id}, _from, state) do
  result =
    case Map.get(state.builtin, tool_id) ||
         Map.get(state.sandbox, tool_id) ||
         Map.get(state.mcp, tool_id) do
      nil  -> {:error, :not_found}
      tool -> {:ok, tool}
    end
  {:reply, result, state}
end
```

Listing 3: Tool lookup across tiers.

The `||` chain is a pattern common in Elixir: it evaluates left-to-right and returns the first non-nil value. This gives builtin tools priority over sandbox tools, and sandbox tools priority over MCP tools. A builtin `web-search` cannot be shadowed by an MCP tool with the same name.

3.5 MCP Registration with Namespacing

When an MCP server is discovered, its tools are registered with a namespace prefix to prevent collisions:

```
def handle_call({:register_mcp, _server_name, _tools}, _from,
                %{frozen: true} = state) do
  {:reply, {:error, :frozen}, state}
end

def handle_call({:register_mcp, server_name, tools}, _from, state) do
  namespaced =
    for tool <- tools, into: %{} do
      namespaced_name = "#{server_name}__#{tool.name}"
      spec = %{
        name: namespaced_name,
        tier: :mcp,
        description: Map.get(tool, :description, ""),
        input_schema: Map.get(tool, :input_schema, %{}),
        output_schema: Map.get(tool, :output_schema, nil),
        validate: build_mcp_validator(tool),
        execute: Map.get(tool, :execute,
                         fn _ -> {:error, :not_implemented} end)
      }
      {namespaced_name, spec}
    end

  Logger.info("[Registry] Registered #{map_size(namespaced)} " <>
              "MCP tools from #{server_name}")
  {:reply, :ok, %{state | mcp: Map.merge(state.mcp, namespaced)}}
end
```

Listing 4: MCP tool registration with namespace and freeze guard.

Note the two clauses of `handle_call`: the first pattern matches on `%{frozen: true}` and rejects the registration. The second clause handles the normal case. This is Elixir's pattern matching applied to state machine design—the frozen guard is not a runtime conditional but a structural match on the state.

3.6 Freeze Semantics: Contract-Locked Configuration

The freeze operation is the registry's most important security feature:

```
def handle_call(:freeze, _from, state) do
  total = map_size(state.builtin) +
          map_size(state.sandbox) +
          map_size(state.mcp)
  Logger.info("[Registry] Frozen with #{total} total tools")
  {:reply, :ok, %{state | frozen: true}}
end
```

Listing 5: Registry freeze operation.

Once frozen, the registry is immutable: no new tools can be registered. This implements the *contract-locked configuration* pattern. When an agent execution begins, the tool set is frozen. The agent cannot discover additional tools or escalate its privileges by connecting to new MCP servers during execution. This is analogous to Capsicum's capability mode (Watson et al., 2010): once a process enters capability mode, it can only use the file descriptors it already holds.

Design Principle 3.2 (Contract Locking)**.** The set of available tools must be frozen before agent execution begins. No tool registration, deregistration, or modification is permitted during execution. This prevents:

1. Privilege escalation through dynamic tool discovery.
2. Time-of-check/time-of-use (TOCTOU) vulnerabilities in capability validation.
3. Non-deterministic behavior from changing tool sets mid-execution.

3.7 Builtin Tool Definitions

Builtin tools are defined as maps with higher-order functions for validation and execution. Here is the `web-search` tool with its input validator:

```
%{
  name: "web-search",
  tier: :builtin,
  description: "Search the web using a query string",
  input_schema: %{
    "type" => "object",
    "properties" => %{
      "query" => %{"type" => "string"},
      "max_results" => %{
        "type" => "integer", "minimum" => 1, "maximum" => 20
      }
    },
    "required" => ["query"]
  },
  output_schema: %{
    "type" => "array",
    "items" => %{"type" => "object"}
  },
  validate: &validate_web_search/1,
  execute: fn input ->
    {:ok, %{results: [], query: input["query"]}}
  end
}
```

Listing 6: Builtin tool definition with validation function.

The `validate` field uses a function capture (`&validate_web_search/1`) that points to a private function performing domain-specific validation:

```
defp validate_web_search(%{"query" => q})
  when is_binary(q) and byte_size(q) > 0, do: :ok
defp validate_web_search(_), do: {:error, :invalid_query}
```

Listing 7: Pattern-matched input validation.

This is pure pattern matching: the function has two clauses. The first matches a map with a non-empty binary `"query"` key and returns `:ok`. The second is a catch-all that returns an error. There is no conditional logic, no null checking, no exception throwing—just exhaustive pattern matching on the input structure.

3.8 URL Safety Validation

Tools that access URLs (`web-scrape`, `pdf-parse`, `git-clone`) share a URL safety validator that blocks access to localhost, loopback, link-local, and private network ranges:

```
defp validate_url_string(url) do
  uri = URI.parse(url)
  blocked_hosts = ["localhost", "127.0.0.1", "0.0.0.0", "::1"]
  blocked_prefixes = [
    "169.254.", "10.", "192.168.",
    "172.16.", "172.17.", "172.18.", "172.19.",
    "172.20.", "172.21.", "172.22.", "172.23.",
    "172.24.", "172.25.", "172.26.", "172.27.",
    "172.28.", "172.29.", "172.30.", "172.31."
  ]
  host = uri.host || ""

  cond do
    host in blocked_hosts ->
      {:error, {:blocked_host, host}}
    Enum.any?(blocked_prefixes,
              &String.starts_with?(host, &1)) ->
      {:error, {:blocked_ip_range, host}}
    true ->
      :ok
  end
end
```

Listing 8: URL safety validation blocking internal addresses.

This prevents Server-Side Request Forgery (SSRF) attacks where an agent tricks a tool into accessing internal infrastructure. The blocked ranges cover the full RFC 1918 private address space plus the AWS metadata endpoint range (`169.254.x.x`).

3.9 Listing and Filtering

The registry supports listing tools filtered by tier, implemented through pattern-matched `handle_call` clauses:

```
def handle_call({:list, nil}, _from, state) do
  all = Map.values(state.builtin) ++
        Map.values(state.sandbox) ++
        Map.values(state.mcp)
  {:reply, all, state}
```

```
end

def handle_call({:list, :builtin}, _from, state) do
  {:reply, Map.values(state.builtin), state}
end

def handle_call({:list, :sandbox}, _from, state) do
  {:reply, Map.values(state.sandbox), state}
end

def handle_call({:list, :mcp}, _from, state) do
  {:reply, Map.values(state.mcp), state}
end
```

Listing 9: Filtered tool listing.

Each tier filter is its own function clause. Adding a new tier would mean adding a new clause—no modification to existing code. This is the open/closed principle realized through pattern matching.

4 Capability Token System

4.1 Design Philosophy

The tool interface layer uses capability-based access control rather than access control lists. Each agent holds unforgeable tokens that grant specific permissions on specific tools. A token is a value: it can be passed, stored, and validated, but it cannot be forged because it carries a cryptographic signature.

Design Principle 4.1 (Capability Tokens as Values)**.** Capability tokens are immutable value types, not references to centralized state. A token encodes:

1. *Who*: the agent identity (`agent_id`).
2. *What*: the tool identity (`tool_id`).
3. *How*: the permitted operations (`permissions`).
4. *How fast*: the maximum invocation rate (`rate_limit`).
5. *Until when*: the expiry time (`expires_at`).
6. *Proof*: the HMAC-SHA256 signature (`signature`).

This design has several advantages over centralized ACLs:

- **No central bottleneck**: Authorization is validated locally by checking the token's signature. There is no database lookup, no network call, no single point of failure.
- **Offline validation**: A token can be validated without contacting any authority—the HMAC signature is self-contained.
- **Fine-grained scoping**: Each token is scoped to exactly one tool. An agent with access to ten tools holds ten tokens.

- **Time-bounded**: Tokens expire automatically. Compromised tokens have a limited window of exploitability.
- **Rate-limited**: Each token enforces a per-minute invocation cap, preventing abuse even with valid credentials.

4.2 Token Structure

The capability token is implemented as an Elixir struct with enforced keys:

```
defmodule ToolInterface.Capability do
  @type permission :: :invoke | :inspect | :compose

  @type t :: %__MODULE__{
    agent_id: String.t(),
    tool_id: String.t(),
    permissions: [permission()],
    rate_limit: pos_integer(),
    expires_at: DateTime.t(),
    signature: binary(),
    invocation_count: non_neg_integer(),
    last_reset_at: DateTime.t()
  }

  @enforce_keys [:agent_id, :tool_id, :permissions,
                 :rate_limit, :expires_at, :signature]
  defstruct [
    :agent_id, :tool_id, :permissions,
    :rate_limit, :expires_at, :signature,
    invocation_count: 0,
    last_reset_at: nil
  ]
end
```

Listing 10: Capability token struct definition.

Three permission levels are defined:

- **`:invoke`** — Permission to execute the tool. This is the most common permission.
- **`:inspect`** — Permission to read the tool's schema and description without executing it.
- **`:compose`** — Permission to use the tool as part of a composed pipeline with other tools.

The `invocation_count` and `last_reset_at` fields track rate limiting state. The rate window resets every 60 seconds.

4.3 Token Creation with HMAC Signing

Token creation validates parameters and computes an HMAC-SHA256 signature over the token's identity fields:

```
@signing_key "tool_interface_capability_signing_key_v1"

def create(agent_id, tool_id, permissions, rate_limit, ttl_seconds)
    when is_binary(agent_id) and is_binary(tool_id)
    and is_list(permissions)
    and is_integer(rate_limit) and rate_limit > 0
    and is_integer(ttl_seconds) and ttl_seconds > 0 do

  valid_permissions = [:invoke, :inspect, :compose]

  if Enum.all?(permissions, &(&1 in valid_permissions)) do
    expires_at = DateTime.add(DateTime.utc_now(),
                              ttl_seconds, :second)
    token = %__MODULE__{
      agent_id: agent_id,
      tool_id: tool_id,
      permissions: permissions,
      rate_limit: rate_limit,
      expires_at: expires_at,
      signature: <<>>,
      invocation_count: 0,
      last_reset_at: DateTime.utc_now()
    }
    signature = compute_signature(token)
    {:ok, %{token | signature: signature}}
  else
    {:error, :invalid_permissions}
  end
end

def create(_, _, _, _, _), do: {:error, :invalid_parameters}
```

Listing 11: Token creation with cryptographic signing.

The guard clause (`when is_binary(agent_id) and ...`) ensures that the function only matches when all parameters have the correct types. If any parameter is wrong—a nil agent ID, a negative rate limit, a non-integer TTL—the second clause matches and returns `{:error, :invalid_parameters}`.

The HMAC signature is computed over a canonical string representation of the token's identity fields:

```
defp compute_signature(%__MODULE__{} = token) do
  payload =
    "#{token.agent_id}:#{token.tool_id}:" <>
    "#{inspect(token.permissions)}:" <>
    "#{token.rate_limit}:" <>
    "#{DateTime.to_iso8601(token.expires_at)}"

  :crypto.mac(:hmac, :sha256, @signing_key, payload)
end
```

Listing 12: HMAC-SHA256 signature computation.

The signing key would come from a secrets manager in production. The signature binds the token's fields together: modifying any field (agent ID, tool ID, permissions, rate limit, or expiry) invalidates the signature.

4.4 Authorization: The Five-Check Pipeline

Token authorization is a pipeline of five checks, implemented as a `cond` expression:

```
def authorize(%__MODULE__{} = token, tool_id) do
  cond do
    token.tool_id != tool_id ->
      {:error, :unauthorized}

    :invoke not in token.permissions ->
      {:error, :unauthorized}

    expired?(token) ->
      {:error, :expired}

    not valid_signature?(token) ->
      {:error, :invalid_signature}

    rate_limited?(token) ->
      {:error, :rate_limited}

    true ->
      updated = increment_invocation(token)
      {:ok, updated}
  end
end

def authorize(_, _), do: {:error, :unauthorized}
```

Listing 13: Five-check authorization pipeline.

The five checks, in order:

1. **Tool match**: The token's `tool_id` must match the requested tool. A token for `web-search` cannot authorize `code-exec`.
2. **Permission check**: The token must include the `:invoke` permission.
3. **Expiry check**: The token must not have passed its `expires_at` time.
4. **Signature verification**: The token's HMAC signature must match a fresh computation, using constant-time comparison to prevent timing attacks:

```
defp valid_signature?(%__MODULE__{} = token) do
  expected = compute_signature(%{token | signature: <<>>})
  :crypto.hash_equals(expected, token.signature)
rescue
  _ -> false
end
```

Listing 14: Constant-time signature verification.

5. **Rate limit check**: The token's invocation count must be below the rate limit for the current 60-second window.

On success, the token is returned with an incremented invocation count. On failure, a specific error atom identifies which check failed, enabling precise error reporting in audit logs.

4.5 Rate Limiting with Sliding Windows

Rate limiting uses a 60-second sliding window:

```
defp rate_limited?(%__MODULE__{} = token) do
  if should_reset_rate_window?(token) do
    false
  else
    token.invocation_count >= token.rate_limit
  end
end

defp should_reset_rate_window?(%__MODULE__{last_reset_at: nil}),
  do: true
defp should_reset_rate_window?(%__MODULE__{last_reset_at: last}) do
  DateTime.diff(DateTime.utc_now(), last, :second) >= 60
end

defp increment_invocation(%__MODULE__{} = token) do
  if should_reset_rate_window?(token) do
    %{token | invocation_count: 1,
              last_reset_at: DateTime.utc_now()}
  else
    %{token | invocation_count: token.invocation_count + 1}
  end
end
```

Listing 15: Rate limiting with window reset.

The rate window resets when more than 60 seconds have elapsed since `last_reset_at`. Note the pattern match on `last_reset_at: nil`—the first invocation always resets the window.

4.6 Cooperative Revocation

Since tokens are values (not references to centralized state), revocation is cooperative: a revoked token is one whose expiry has been set to the past.

```
def revoke(%__MODULE__{} = token) do
  expired_token = %{token |
    expires_at: DateTime.add(DateTime.utc_now(), -1, :second)
  }
  signature = compute_signature(expired_token)
  %{expired_token | signature: signature}
end
```

Listing 16: Cooperative token revocation.

The revoked token's signature is recomputed so that it remains structurally valid but immediately fails the expiry check. This approach is appropriate for short-lived tokens (the default TTL is 3600 seconds). For long-lived tokens in a production system, a revocation list would supplement cooperative revocation.

5 Sandboxed Execution

5.1 Isolation Through Process Boundaries

The sandbox module executes tool operations in isolated BEAM processes. Each execution spawns a new process with its own heap, executes the tool's function, and sends the result back to the parent via message passing. The parent monitors the spawned process and kills it if it exceeds the timeout.

Design Principle 5.1 (Process-Level Isolation)**.** Each sandboxed execution runs in a spawned BEAM process that:

1. Has its own heap—no shared mutable state with the parent.
2. Communicates only through message passing—no direct memory access.
3. Is monitored—the parent receives a `:DOWN` message if the process crashes.
4. Is time-bounded—the parent kills the process after a configurable timeout.

This provides the same isolation guarantee as E2B ephemeral sandboxes, but at the language-runtime level rather than the container level.

5.2 The Execute Function

The sandbox executor is the most carefully designed function in the system:

```
defmodule ToolInterface.Sandbox do
  require Logger

  @default_timeout 600_000    # 10 minutes (matching E2B)
  @max_timeout 1_800_000      # 30 minutes absolute maximum

  def execute(tool_spec, input, opts \\ []) do
    timeout = min(
      Keyword.get(opts, :timeout, @default_timeout),
      @max_timeout
    )
    parent = self()
    ref = make_ref()

    {pid, monitor_ref} =
      spawn_monitor(fn ->
        # This process has its own heap.
        # No shared mutable state with parent.
        result =
          try do
            case tool_spec.execute.(input) do
              {:ok, _} = ok  -> ok
              {:error, _} = err -> err
              other -> {:ok, other}
            end
          rescue
            e -> {:error, {:execution_failed,
                           Exception.message(e)}}
          catch
```

```
30              :exit, reason ->
31                {:error, {:sandbox_exit, reason}}
32              :throw, value ->
33                {:error, {:sandbox_throw, value}}
34            end
35
36          send(parent, {:sandbox_result, ref, result})
37        end)
38
39      receive do
40        {:sandbox_result, ^ref, result} ->
41          Process.demonitor(monitor_ref, [:flush])
42          result
43
44        {:DOWN, ^monitor_ref, :process, ^pid, reason} ->
45          {:error, {:sandbox_crashed, reason}}
46      after
47        timeout ->
48          Process.exit(pid, :kill)
49          receive do
50            {:DOWN, ^monitor_ref, :process, ^pid, _} -> :ok
51          after
52            5_000 ->
53              Process.demonitor(monitor_ref, [:flush])
54          end
55          {:error, :timeout}
56      end
57    end
58  end
```

Listing 17: Complete sandbox executor with monitored process isolation.

This function deserves a detailed walkthrough:

1. **Timeout clamping** (lines 8–11): The requested timeout is clamped to an absolute maximum of 30 minutes. Even if a caller requests an infinite timeout, the sandbox will terminate.

2. **Reference creation** (line 13): A unique reference (`make_ref()`) tags the result message. This prevents a stale result from a previous sandbox execution from being confused with the current one.

3. **Spawn with monitor** (lines 15–33): `spawn_monitor/1` atomically creates a new process and establishes a monitor. If the process crashes, the parent receives a `{:DOWN, monitor_ref, :process, pid, reason}` message.

4. **Comprehensive error handling** (lines 19–31): The spawned process wraps the tool execution in a `try/rescue/catch` block that handles:

   - Normal `{:ok, _}` and `{:error, _}` returns (passed through).
   - Unexpected return values (wrapped in `{:ok, other}`).
   - Raised exceptions (caught and wrapped as `{:error, {:execution_failed, msg}}`).
   - Process exits (caught as `{:error, {:sandbox_exit, reason}}`).

- Throws (caught as `{:error, {:sandbox_throw, value}}`).

5. **Result collection** (lines 35–37): The parent waits for the sandbox result, pattern matching on the unique reference. On success, the monitor is flushed to avoid a spurious `:DOWN` message.

6. **Crash handling** (lines 39–40): If the process crashes before sending a result, the monitor's `:DOWN` message is received and converted to an error tuple.

7. **Timeout handling** (lines 41–49): If neither a result nor a crash arrives within the timeout, the parent kills the sandbox process with `Process.exit(pid, :kill)`, waits for the `:DOWN` message to confirm the kill, and returns `{:error, :timeout}`.

5.3 Parallel Sandbox Execution

Multiple tool operations can execute in parallel sandboxes using `Task.async_stream`:

```
def execute_parallel(tool_input_pairs, opts \\ []) do
  max_concurrency = Keyword.get(opts, :max_concurrency, 5)
  timeout = Keyword.get(opts, :timeout, @default_timeout)

  tool_input_pairs
  |> Task.async_stream(
    fn {tool_spec, input} ->
      execute(tool_spec, input, timeout: timeout)
    end,
    max_concurrency: max_concurrency,
    timeout: timeout + 5_000,
    on_timeout: :kill_task
  )
  |> Enum.map(fn
    {:ok, result}       -> result
    {:exit, :timeout}   -> {:error, :timeout}
    {:exit, reason}     -> {:error, {:task_failed, reason}}
  end)
end
```

Listing 18: Parallel sandbox execution with controlled concurrency.

The `max_concurrency` option bounds the number of simultaneous sandbox processes, preventing resource exhaustion. The Task timeout is set slightly longer than the sandbox timeout (5 seconds longer) to allow the sandbox's internal cleanup to complete before the Task layer forcibly terminates.

5.4 URL Validation for Sandbox Tools

Sandbox tools that access external URLs share the same URL safety validation used by builtin tools, reused through a public API:

```
@spec validate_url(String.t()) ::
  :ok | {:error, {:blocked_url, String.t()}}
def validate_url(url) when is_binary(url) do
  uri = URI.parse(url)
  host = uri.host || ""

```

```
  blocked_hosts = ["localhost", "127.0.0.1",
                   "0.0.0.0", "::1", "[::1]"]
  blocked_prefixes = [
    "169.254.", "10.", "192.168.",
    "172.16.", "172.17.", "172.18.", "172.19.",
    "172.20.", "172.21.", "172.22.", "172.23.",
    "172.24.", "172.25.", "172.26.", "172.27.",
    "172.28.", "172.29.", "172.30.", "172.31."
  ]

  cond do
    host in blocked_hosts ->
      {:error, {:blocked_url, "Host #{host} is blocked"}}
    Enum.any?(blocked_prefixes,
              &String.starts_with?(host, &1)) ->
      {:error, {:blocked_url, "IP range #{host} is blocked"}}
    true ->
      :ok
  end
end

def validate_url(_), do: {:error, {:blocked_url, "Invalid URL"}}
```

Listing 19: URL validation reused across tiers.

6 MCP Protocol Client

6.1 The Model Context Protocol

The Model Context Protocol (MCP) (MCP Specification, 2024) defines a standard interface for language models to discover and invoke tools provided by external servers. In the AGENT-HERO production platform, MCP integration uses the `@modelcontextprotocol/sdk` library with `StreamableHTTPClientTransport` for efficient HTTP-based communication. The Elixir reference implementation mirrors this design using Finch for HTTP and Jason for JSON encoding.

An MCP server exposes three operations:

1. **Initialize** — Establish a session. The server returns a session ID in the `mcp-session-id` response header.
2. **List Tools** — Returns an array of tool descriptors, each with a name, description, and JSON Schema for its input.
3. **Call Tool** — Invokes a specific tool with typed arguments and returns a structured result.

All three operations use JSON-RPC 2.0 over HTTP POST, with Bearer token authentication.

6.2 Session Initialization

The MCP client begins by establishing a session with the server:

```
defp initialize_session(url, token) do
  body = Jason.encode!(%{
    jsonrpc: "2.0",
    id: generate_request_id(),
    method: "initialize",
    params: %{
      protocolVersion: "2024-11-05",
      capabilities: %{},
      clientInfo: %{
        name: "tool_interface",
        version: "0.1.0"
      }
    }
  })

  case http_post(url, body, auth_headers(token, nil),
                 @discovery_timeout) do
    {:ok, %{status: 200, headers: headers}} ->
      session_id = extract_session_id(headers)
      {:ok, session_id}
    {:ok, %{status: status}} ->
      {:error, {:http_error, status}}
    {:error, reason} ->
      {:error, {:connection_failed, reason}}
  end
end
```

Listing 20: MCP session initialization via JSON-RPC.

The session ID is extracted from the response headers and included in all subsequent requests. This allows the server to maintain per-session state (tool availability, rate limits, etc.).

6.3 Tool Discovery and Registration

After session initialization, the client lists available tools and registers them:

```
def discover_and_register(server_name, url, token) do
  Logger.info("[MCPClient] Discovering tools from " <>
              "#{server_name} at #{url}")

  with :ok <- validate_server_url(url),
       {:ok, session_id} <- initialize_session(url, token),
       {:ok, raw_tools} <- list_tools(url, token, session_id) do

    tools = Enum.map(raw_tools, fn raw_tool ->
      %{
        name: raw_tool["name"],
        description: raw_tool["description"] || "",
        input_schema: raw_tool["inputSchema"] || %{},
        execute: build_remote_executor(
          url, token, session_id, raw_tool["name"]
        )
      }
    end)

```

```
    Logger.info("[MCPClient] Discovered #{length(tools)} " <>
                "tools from #{server_name}")
    ToolInterface.Registry.register_mcp(server_name, tools)
  end
end
```

Listing 21: MCP tool discovery and registration pipeline.

The `with` expression chains three operations: URL validation, session initialization, and tool listing. If any step fails, the entire pipeline short-circuits and returns the error. This is the Elixir equivalent of monadic `bind` in Haskell or the `Result` chain in Rust.

6.4 Closures as Remote Executors

The most elegant part of the MCP client is the `build_remote_executor` function:

```
defp build_remote_executor(url, token, session_id, tool_name) do
  fn input ->
    call_tool(url, token, session_id, tool_name, input)
  end
end
```

Listing 22: Closure-based remote executor construction.

This function returns a *closure*—an anonymous function that captures the server URL, authentication token, session ID, and tool name in its environment. The closure has the same type signature as any other tool's `execute` function: `fn input -> {:ok, result} | {:error, reason} end`. The caller—the registry, the sandbox, the invocation pipeline—does not know or care that this function dispatches over HTTP. The network boundary is hidden behind a function value.

This is the key insight: by representing remote tool invocation as a closure, MCP tools are compositionally identical to local tools. They can be stored in the registry, authorized via capability tokens, audited, and composed into pipelines, all using the same code paths.

6.5 Remote Tool Invocation

The `call_tool` function sends a JSON-RPC request and parses the response:

```
def call_tool(url, token, session_id, tool_name, input) do
  body = Jason.encode!(%{
    jsonrpc: "2.0",
    id: generate_request_id(),
    method: "tools/call",
    params: %{name: tool_name, arguments: input}
  })

  case http_post(url, body,
                 auth_headers(token, session_id),
                 @invocation_timeout) do
    {:ok, %{status: 200, body: response_body}} ->
      case Jason.decode(response_body) do
        {:ok, %{"result" => result}} ->
          {:ok, result}
```

```
        {:ok, %{"error" => %{"message" => msg}}} ->
          {:error, {:mcp_error, msg}}
        {:error, _} ->
          {:error, :invalid_response}
      end
    {:ok, %{status: status}} ->
      {:error, {:http_error, status}}
    {:error, reason} ->
      {:error, {:connection_failed, reason}}
  end
end
```

Listing 23: JSON-RPC tool invocation.

Pattern matching on the nested JSON response provides exhaustive error handling: a successful result, a JSON-RPC error, a malformed response, an HTTP error, and a connection failure each produce a distinct, tagged error tuple.

6.6 Parallel Multi-Server Discovery

The `discover_all` function connects to multiple MCP servers in parallel:

```
def discover_all(servers, opts \\ []) do
  max_concurrency = Keyword.get(opts, :max_concurrency, 10)
  timeout = Keyword.get(opts, :timeout, @discovery_timeout)

  servers
  |> Task.async_stream(
    fn server ->
      case discover_and_register(
        server.name, server.url, server.token
      ) do
        :ok -> {:ok, server.name}
        {:error, reason} ->
          Logger.warning(
            "[MCPClient] Failed to discover from " <>
            "#{server.name}: #{inspect(reason)}"
          )
          {:error, server.name, reason}
      end
    end,
    max_concurrency: max_concurrency,
    timeout: timeout,
    on_timeout: :kill_task
  )
  |> Enum.map(fn
    {:ok, result}       -> result
    {:exit, :timeout}   -> {:error, "unknown", :timeout}
    {:exit, reason}     -> {:error, "unknown", {:task_failed, reason}}
  end)
end
```

Listing 24: Parallel MCP discovery with fault isolation.

This mirrors the AGENT-HERO platform's `Promise.allSettled()` pattern: each server is discovered independently, and a slow or failing server does not block discovery from other servers. The `max_concurrency: 10` bound prevents overwhelming the

network or the BEAM scheduler with too many simultaneous HTTP connections.

6.7 Authentication Headers

Bearer token authentication is implemented through header construction:

```
defp auth_headers(token, session_id) do
  headers = [
    {"content-type", "application/json"},
    {"authorization", "Bearer #{token}"},
    {"accept", "application/json"}
  ]

  if session_id do
    [{"mcp-session-id", session_id} | headers]
  else
    headers
  end
end
```

Listing 25: Authentication header construction with session ID.

The session ID is prepended to the headers list only when present. This is idiomatic Elixir: the `if` expression returns the augmented or original list, and the caller receives the correct headers without conditional logic at the call site.

7 The Invocation Pipeline: Facade Module

7.1 Composing the Components

The `ToolInterface` facade module composes the registry, capability system, sandbox, and audit logger into a single invocation pipeline:

```
defmodule ToolInterface do
  alias ToolInterface.{Registry, Capability, Sandbox, Audit}

  @spec invoke(String.t(), String.t(), map(), Capability.t()) ::
    {:ok, any()} | {:error, term()}
  def invoke(agent_id, tool_id, input, capability_token) do
    start_time = System.monotonic_time(:millisecond)

    with {:ok, _token} <- Capability.authorize(
                            capability_token, tool_id),
         {:ok, tool_spec} <- Registry.lookup(tool_id),
         :ok <- validate_input(tool_spec, input) do
      result = execute_tool(tool_spec, input)
      duration = System.monotonic_time(:millisecond) - start_time
      status = if match?({:ok, _}, result), do: :ok, else: :error

      Audit.log_invocation(
        agent_id, tool_id, input, result, duration, status
      )
      result
    else
      {:error, _reason} = error ->
```

```
        duration = System.monotonic_time(:millisecond) - start_time
        Audit.log_invocation(
          agent_id, tool_id, input, error, duration, :error
        )
        error
    end
  end
end
```

Listing 26: Complete tool invocation pipeline.

The `with` expression is the heart of the invocation pipeline. It chains three operations:

1. **Authorize**: Check the capability token against the requested tool. If the token is invalid, expired, rate-limited, or scoped to a different tool, short-circuit with the error.
2. **Lookup**: Find the tool specification in the registry. If the tool does not exist, short-circuit with `{:error, :not_found}`.
3. **Validate**: Run the tool's input validator (if present). If validation fails, short-circuit with the validation error.

If all three succeed, execution proceeds. The result (success or failure) and execution duration are recorded in the audit log. The `else` clause handles any short-circuit error, also logging it for audit purposes.

7.2 Tier-Aware Execution Dispatch

Tool execution is dispatched based on the tool's tier:

```
defp execute_tool(%{tier: :sandbox} = tool_spec, input) do
  Sandbox.execute(tool_spec, input)
end

defp execute_tool(tool_spec, input) do
  try do
    tool_spec.execute.(input)
  rescue
    e -> {:error, {:execution_failed, Exception.message(e)}}
  end
end
```

Listing 27: Tier-aware execution dispatch via pattern matching.

Two function clauses implement the dispatch:

- **Sandbox tools**: Pattern match on `%{tier: :sandbox}` and delegate to `Sandbox.execute/2`, which spawns an isolated process.
- **All other tools**: Call the tool's `execute` function directly in the current process. Exceptions are caught and wrapped in error tuples.

For MCP tools (tier `:mcp`), the second clause applies: the tool's `execute` function is the closure built by `build_remote_executor`, which dispatches over HTTP. The facade does not need to know this—it simply calls the function.

7.3 Input Validation

Input validation uses the tool's optional `validate` function:

```
defp validate_input(tool_spec, input) do
  case tool_spec.validate do
    nil ->
      :ok
    validate_fn when is_function(validate_fn, 1) ->
      case validate_fn.(input) do
        :ok              -> :ok
        {:ok, _validated} -> :ok
        {:error, _} = err -> err
      end
  end
end
```

Listing 28: Input validation with optional validator.

The pattern match on `nil` versus `is_function(validate_fn, 1)` provides a clean optional validation path. When the validator is nil, all inputs pass. When present, the validator is called and its result is pattern-matched to normalize the return value.

7.4 Public API Functions

The facade exposes four additional functions:

```
def grant_capability(agent_id, tool_id, opts \\ []) do
  permissions = Keyword.get(opts, :permissions, [:invoke])
  rate_limit = Keyword.get(opts, :rate_limit, 60)
  ttl = Keyword.get(opts, :ttl_seconds, 3600)
  Capability.create(agent_id, tool_id, permissions,
                    rate_limit, ttl)
end

def discover_mcp(server_name, url, token) do
  ToolInterface.MCPClient.discover_and_register(
    server_name, url, token
  )
end

def freeze do
  Registry.freeze()
end

def list_tools(tier \\ nil) do
  Registry.list(tier)
end
```

Listing 29: Facade public API.

The facade provides sensible defaults: 60 invocations per minute, 1 hour TTL, invoke-only permissions. A caller can override any of these through keyword options.

7.5 OTP Application and Supervision

The system starts as an OTP application with a supervision tree:

```
defmodule ToolInterface.Application do
  use Application

  @impl true
  def start(_type, _args) do
    children = [
      ToolInterface.Registry,
      ToolInterface.Audit
    ]
    opts = [strategy: :one_for_one,
            name: ToolInterface.Supervisor]
    Supervisor.start_link(children, opts)
  end
end
```

Listing 30: OTP supervision tree.

The `:one_for_one` strategy means that if the Registry crashes, only the Registry is restarted—the Audit logger continues running. If the Audit logger crashes, only the Audit logger is restarted. This fault isolation ensures that a bug in audit logging cannot take down tool registration, and vice versa.

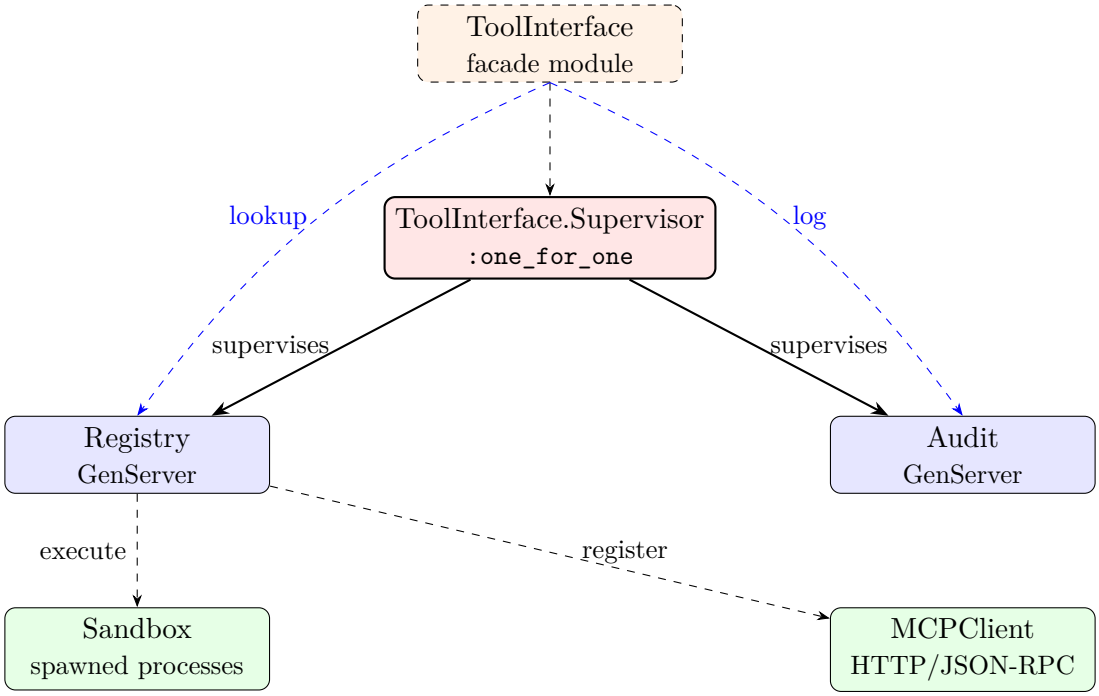

Figure 2: OTP supervision tree. Solid arrows represent supervision relationships. Dashed arrows represent function calls and message passing. The facade module is not a process—it coordinates calls to the supervised GenServers.

8 Audit Logger

8.1 Design: Asynchronous, Non-Blocking Logging

The audit logger records every tool invocation with full context. It is implemented as a GenServer that receives log entries asynchronously via `GenServer.cast/2`, ensuring that audit overhead does not impact tool invocation latency.

Design Principle 8.1 (Non-Blocking Audit)**.** Audit logging must not add latency to tool invocations. The audit logger receives entries via asynchronous casts (fire-and-forget). Entries are accumulated in memory and periodically flushed to persistent storage.

8.2 Entry Structure

Each audit entry captures the complete invocation context:

```
defmodule ToolInterface.Audit do
  use GenServer
  require Logger

  @type entry :: %{
    id: String.t(),            # unique entry ID
    timestamp: DateTime.t(),    # UTC timestamp
    agent_id: String.t(),       # who invoked
    tool_id: String.t(),        # what was invoked
    input: map(),               # sanitized input
    output: any(),              # sanitized output
    duration_ms: non_neg_integer(),  # execution time
    status: :ok | :error        # outcome
  }
end
```

Listing 31: Audit entry structure.

8.3 Asynchronous Logging with Telemetry

The `log_invocation` function constructs an entry, emits a telemetry event, and casts to the GenServer:

```
def log_invocation(agent_id, tool_id, input, output,
                   duration_ms, status) do
  entry = %{
    id: generate_entry_id(),
    timestamp: DateTime.utc_now(),
    agent_id: agent_id,
    tool_id: tool_id,
    input: sanitize_for_logging(input),
    output: sanitize_for_logging(output),
    duration_ms: duration_ms,
    status: status
  }

  # Emit telemetry event for external observability
  :telemetry.execute(
```

```
    [:tool_interface, :invocation],
    %{duration_ms: duration_ms},
    %{agent_id: agent_id, tool_id: tool_id, status: status}
  )

  GenServer.cast(__MODULE__, {:log, entry})
end
```

Listing 32: Audit logging with telemetry integration.

The `:telemetry.execute/3` call emits a structured event that can be consumed by Prometheus exporters, StatsD clients, or any other telemetry backend. This integrates with the broader Erlang/Elixir observability ecosystem without coupling the audit logger to a specific monitoring system.

8.4 Sensitive Data Sanitization

Before logging, input and output data are sanitized to prevent credential leakage:

```
defp sanitize_for_logging(data) when is_map(data) do
  sensitive_keys = ["password", "token", "secret",
                    "api_key", "authorization"]

  Map.new(data, fn
    {key, _value} when is_binary(key) ->
      if String.downcase(key) in sensitive_keys do
        {key, "[REDACTED]"}
      else
        {key, sanitize_for_logging(Map.get(data, key))}
      end
    {key, value} ->
      {key, sanitize_for_logging(value)}
  end)
end

defp sanitize_for_logging(data) when is_list(data) do
  Enum.map(data, &sanitize_for_logging/1)
end

defp sanitize_for_logging(data), do: data
```

Listing 33: Recursive sensitive data sanitization.

The sanitizer recursively traverses maps and lists, replacing values at sensitive keys with `[REDACTED]`. This prevents Bearer tokens, API keys, and passwords from appearing in audit logs.

8.5 Ring Buffer with Capacity Bounds

The GenServer maintains entries in a bounded ring buffer:

```
defstruct entries: [],
          entry_count: 0,
          max_entries: 10_000,
          flush_interval_ms: 60_000

```

```
@impl true
def handle_cast({:log, entry}, state) do
  Logger.debug(
    "[Audit] #{entry.agent_id} -> #{entry.tool_id} " <>
    "[#{entry.status}] #{entry.duration_ms}ms"
  )

  entries =
    if state.entry_count >= state.max_entries do
      # Drop oldest entry when at capacity
      [entry | Enum.take(state.entries, state.max_entries - 1)]
    else
      [entry | state.entries]
    end

  new_count = min(state.entry_count + 1, state.max_entries)
  {:noreply, %{state | entries: entries, entry_count: new_count}}
end
```

Listing 34: Ring buffer entry storage with capacity bounds.

When the buffer reaches capacity (default 10,000 entries), the oldest entry is dropped. This prevents unbounded memory growth in long-running systems. Periodic flushing to persistent storage ensures that entries survive process restarts.

8.6 Periodic Flushing

The audit GenServer schedules periodic flushes using `Process.send_after`:

```
@impl true
def init(opts) do
  max_entries = Keyword.get(opts, :max_entries, 10_000)
  flush_interval = Keyword.get(opts, :flush_interval_ms, 60_000)

  state = %__MODULE__{
    entries: [],
    entry_count: 0,
    max_entries: max_entries,
    flush_interval_ms: flush_interval
  }

  if flush_interval > 0 do
    Process.send_after(self(), :periodic_flush, flush_interval)
  end

  {:ok, state}
end

@impl true
def handle_info(:periodic_flush, state) do
  if state.entry_count > 0 do
    do_flush(state.entries)
  end
  Process.send_after(self(), :periodic_flush,
                     state.flush_interval_ms)
  {:noreply, state}
end
```

Listing 35: Periodic flush scheduling.

The flush mechanism uses `handle_info` to process the self-sent `:periodic_flush` message. In production, `do_flush` would write to a database, object storage, or streaming pipeline (Kafka, CloudWatch, etc.). The reference implementation retains entries in memory only.

8.7 Query and Statistics

The audit logger supports querying entries and computing aggregate statistics:

```
defp compute_stats(entries) do
  total = length(entries)
  ok_count = Enum.count(entries, &(&1.status == :ok))
  error_count = total - ok_count
  total_duration = Enum.reduce(entries, 0,
                               &(&1.duration_ms + &2))
  avg_duration = if total > 0,
                    do: total_duration / total,
                    else: 0.0

  %{
    total: total,
    ok: ok_count,
    error: error_count,
    avg_duration_ms: Float.round(avg_duration, 2),
    tools_used: entries
                |> Enum.map(& &1.tool_id)
                |> Enum.uniq()
                |> length(),
    agents_active: entries
                   |> Enum.map(& &1.agent_id)
                   |> Enum.uniq()
                   |> length()
  }
end
```

Listing 36: Audit statistics computation.

The pipeline operator makes the data flow readable: take the entries, extract tool IDs, remove duplicates, count. This computes six aggregate metrics: total invocations, success count, error count, average duration, unique tools used, and active agents.

9 Production Validation: Mapping to Agent-Hero

The reference implementation's design maps directly to the Agent-Hero production platform. This section traces the correspondence between the Elixir reference implementation and the deployed TypeScript/Next.js system.

9.1 Three-Tier Model Correspondence

Tier	Elixir Reference	Agent-Hero Production
Builtin	Static tool specs in `Registry.init/1`	Vercel AI SDK `tool()` definitions with Zod schemas
Sandbox	`Sandbox.execute/3` spawning isolated BEAM processes	E2B Code Interpreter with 10-minute ephemeral sandboxes
MCP	`MCPClient.discover_and_register/3` via Finch HTTP	`StreamableHTTPClientTransport` from `@modelcontextprotocol/sdk`

9.2 Capability Token Correspondence

In the Elixir implementation, capability tokens are HMAC-signed structs with TTL, rate limiting, and permission scoping. In Agent-Hero, the same security model is implemented through per-agent tool permissions stored in the database:

```
// Agent-Hero: per-agent tool permissions
const frozenTools = Object.freeze(
  resolveToolsForAgent(agent, contract)
);

// Tool set is immutable during execution
async function executeAgent(agent, task) {
  const tools = frozenTools;
  return await runAgentLoop(agent, task, tools);
}
```

Listing 37: Agent-Hero tool permission model (simplified).

The `Object.freeze()` in TypeScript corresponds to `Registry.freeze()` in Elixir. Both ensure that the tool set is immutable during agent execution.

9.3 Sandbox Correspondence

The Elixir sandbox uses BEAM process isolation with `spawn_monitor` and hard timeouts. The Agent-Hero platform uses E2B sandboxes with the same 10-minute timeout:

```
import { Sandbox } from '@e2b/code-interpreter';

async function executeSandboxed(code: string, language: string) {
  const sandbox = await Sandbox.create({
    timeoutMs: 10 * 60 * 1000,  // 10 minutes
  });
  try {
    const result = await sandbox.runCode(code, { language });
    return {
      stdout: result.logs.stdout,
      stderr: result.logs.stderr,
      exitCode: result.exitCode,
    };
  } finally {
    await sandbox.close();  // ephemeral: always clean up
  }
}
```

Listing 38: E2B sandbox execution in Agent-Hero.

The correspondence is direct: `spawn_monitor` maps to `Sandbox.create`, the `after timeout` clause maps to `timeoutMs`, and `Process.exit(pid, :kill)` maps to the implicit sandbox termination.

9.4 MCP Correspondence

The Elixir MCP client's `discover_all` with `Task.async_stream` maps to AGENT-HERO's parallel server connection:

```
const connections = await Promise.allSettled(
  mcpServers.map(async (server) => {
    const transport = new StreamableHTTPClientTransport(
      new URL(server.url),
      { headers: { Authorization: `Bearer ${server.token}` } }
    );
    const client = new Client({
      name: "agent-hero", version: "1.0"
    });
    await client.connect(transport);
    const { tools } = await client.listTools();
    return tools.map(t => ({
      name: `${server.name}__${t.name}`,
      description: t.description,
      schema: t.inputSchema,
    }));
  })
);
```

Listing 39: Parallel MCP discovery in Agent-Hero.

`Task.async_stream` with `on_timeout: :kill_task` provides the same semantics as `Promise.allSettled`: each server is handled independently, failures are isolated, and the system proceeds with whatever tools were successfully discovered.

9.5 Audit Correspondence

The Elixir audit logger's telemetry integration maps to AGENT-HERO's credit-based billing system, where each tool invocation consumes credits proportional to execution time and API costs. The audit entry's `duration_ms` field directly feeds the billing computation.

10 Design Analysis and Comparisons

10.1 Comparison to Classical Operating System Patterns

Classical OS Concept	AI OS Equivalent	Elixir Implementation
File descriptor	Capability token	`Capability.t()` struct
Device driver	Tool implementation	`tool_spec.execute` function
System call	Tool invocation	`ToolInterface.invoke/4`
`/dev` directory	Tool registry	`Registry` GenServer
Major/minor numbers	Tier + tool ID	`:builtin` | `:sandbox` | `:mcp` + string
`ioctl` parameters	Tool input schema	JSON Schema map
Kernel ring 0	Builtin tier	Direct function execution
User-space ring 3	Sandbox tier	Spawned BEAM process
Network driver	MCP tier	HTTP/JSON-RPC closure
`open()` permission	Capability authorization	`Capability.authorize/2`
`close()` cleanup	Token revocation	`Capability.revoke/1`

The key difference is that the AI OS must handle *dynamic* tool discovery (through MCP) whereas Unix device discovery is largely static. The freeze mechanism bridges this gap: tools are discovered dynamically but frozen at execution time, providing the stability guarantees of static configuration.

10.2 Comparison to Plan 9

Plan 9's "everything is a file" philosophy (Pike et al., 1995) suggests that the AI OS should present all tools through a uniform interface. MCP achieves this: every tool, regardless of implementation, is invoked through the same JSON-RPC protocol with the same schema-based input/output contract.

Plan 9 goes further with *per-process name spaces*: each process sees a different file system tree. The AI OS analogue is per-agent capability scoping: each agent's tokens define a different view of the tool registry, restricted to the tools the agent is authorized to use.

10.3 Comparison to Capsicum and seL4

Capsicum (Watson et al., 2010) and seL4 (Klein et al., 2009) provide the closest security analogues:

- **Capsicum's capability mode** is directly analogous to registry freezing. Once an execution begins, no new capabilities can be acquired.
- **seL4's per-syscall capabilities** are analogous to per-invocation token authorization. Every tool call requires a valid capability token.
- **seL4's formal verification** is a goal for future work. The current system relies on Elixir's type specifications and runtime validation rather than machine-checked proofs.

10.4 Functional Composition Properties

The tool interface design inherits several useful properties from functional programming:

Proposition 10.1 (Composition Associativity)**.** *For any three tools f, g, h whose types align (the output type of f matches the input type of g, and the output type of g matches the input type of h), the composition is associative:*

$$x \mid\!> f \mid\!> g \mid\!> h = x \mid\!> f \mid\!> (g \mid\!> h) = x \mid\!> (f \mid\!> g) \mid\!> h$$

This follows directly from the fact that tool execution is function application, and function composition is associative. In practical terms, it means that tool pipelines can be freely refactored—extracting a sub-pipeline into a named composition, or inlining a named composition into a larger pipeline—without changing behavior.

Proposition 10.2 (Type-Safe Composition)**.** *If tool f produces output conforming to schema σ_B and tool g accepts input conforming to schema σ_B, then the composition $f \mid\!> g$ preserves type correctness: valid input to f produces valid input to g.*

This is enforced at runtime through the validation pipeline: each tool's `validate` function checks its input before execution. Schema-level composition checking (verifying that output schemas match input schemas before execution begins) is a natural extension for future work.

10.5 Security Properties

Proposition 10.3 (Sandbox Isolation)**.** *If two agents a and b have disjoint capability sets (no shared tokens), and all sandbox executions run in separate BEAM processes, then no direct information flow exists between a's tool executions and b's tool executions.*

This follows from BEAM process isolation: each process has its own heap, and inter-process communication requires explicit message passing. A sandbox process for agent a cannot read the memory of a sandbox process for agent b, and the capability system prevents a from invoking tools authorized only for b.

Remark 10.4 (Limitations)**.** This isolation guarantee does not address timing side channels (observable differences in execution duration), resource-usage side channels (observable memory or CPU consumption), or prompt injection attacks (external content containing adversarial instructions). These require additional countermeasures beyond the tool interface layer.

10.6 Scalability Properties

The BEAM virtual machine provides strong scalability properties for the tool interface:

- **Concurrent invocations**: The BEAM scheduler handles millions of lightweight processes, enabling massive concurrent tool execution.
- **Fault isolation**: A crashing tool affects only its own process. The registry, audit logger, and other tool executions are unaffected.

- **Back-pressure**: The GenServer message queue provides natural back-pressure. If the registry is overwhelmed with lookup requests, callers block on `GenServer.call`, creating upstream pressure.
- **Distribution**: OTP distribution allows the registry to span multiple nodes. Capability tokens are validated locally (HMAC verification requires only the signing key, not a database lookup).

11 Case Studies

11.1 Web Scraping Pipeline

Consider an agent tasked with extracting structured data from the web. The pipeline composes three builtin tools:

```
# Grant capabilities for each tool in the pipeline
{:ok, search_cap} = ToolInterface.grant_capability(
  "agent-1", "web-search", permissions: [:invoke])
{:ok, scrape_cap} = ToolInterface.grant_capability(
  "agent-1", "web-scrape", permissions: [:invoke])
{:ok, transform_cap} = ToolInterface.grant_capability(
  "agent-1", "text-transform", permissions: [:invoke])

# Step 1: Search
{:ok, urls} = ToolInterface.invoke(
  "agent-1", "web-search",
  %{"query" => "elixir otp patterns"}, search_cap)

# Step 2: Scrape (URL safety validated automatically)
{:ok, html} = ToolInterface.invoke(
  "agent-1", "web-scrape",
  %{"url" => hd(urls.results)}, scrape_cap)

# Step 3: Transform
{:ok, structured} = ToolInterface.invoke(
  "agent-1", "text-transform",
  %{"text" => html.content, "operation" => "extract"},
  transform_cap)
```

Listing 40: Web scraping pipeline composing three tools.

Each invocation goes through the full pipeline: capability authorization, registry lookup, input validation (including URL safety for the scrape step), execution, and audit logging. The URL safety validator blocks any attempt to scrape `localhost` or private IP ranges, preventing SSRF attacks.

11.2 Sandboxed Code Execution

Code execution tools run in isolated BEAM processes:

```
{:ok, exec_cap} = ToolInterface.grant_capability(
  "agent-2", "code-exec",
  permissions: [:invoke], rate_limit: 10)

```

```
# Executes in a separate BEAM process with 10-min timeout
{:ok, result} = ToolInterface.invoke(
  "agent-2", "code-exec",
  %{"code" => "Enum.sum(1..100)", "language" => "elixir"},
  exec_cap)

# If execution exceeds timeout:
# {:error, :timeout}

# If code raises an exception:
# {:error, {:execution_failed, "ArithmeticError: ..."}}

# If sandbox process crashes:
# {:error, {:sandbox_crashed, :normal}}
```

Listing 41: Sandboxed code execution with timeout.

The rate limit of 10 invocations per minute prevents an agent from spawning too many sandbox processes. Each error case produces a distinct tagged tuple, enabling precise error handling and reporting.

11.3 Multi-MCP Server Discovery

At execution startup, the platform discovers tools from multiple MCP servers:

```
servers = [
  %{name: "github", url: "https://mcp.github.com",
    token: "ghp_..."},
  %{name: "slack", url: "https://mcp.slack.com",
    token: "xoxb-..."},
  %{name: "jira", url: "https://mcp.jira.com",
    token: "jira_..."}
]

# Discover in parallel - failures are isolated
results = ToolInterface.MCPClient.discover_all(servers,
  max_concurrency: 10, timeout: 30_000)

# results might be:
# [{:ok, "github"}, {:ok, "slack"},
#  {:error, "jira", :connection_refused}]

# Freeze the registry - no more tools can be added
ToolInterface.freeze()

# Agent can now use: github__create_issue, slack__send_message
# but NOT any jira tools (discovery failed)
# and cannot discover new servers (registry frozen)
```

Listing 42: Multi-server MCP discovery and freeze.

The parallel discovery ensures that a slow Jira server does not delay GitHub and Slack tool availability. After freezing, the tool set is immutable for the duration of the agent execution.

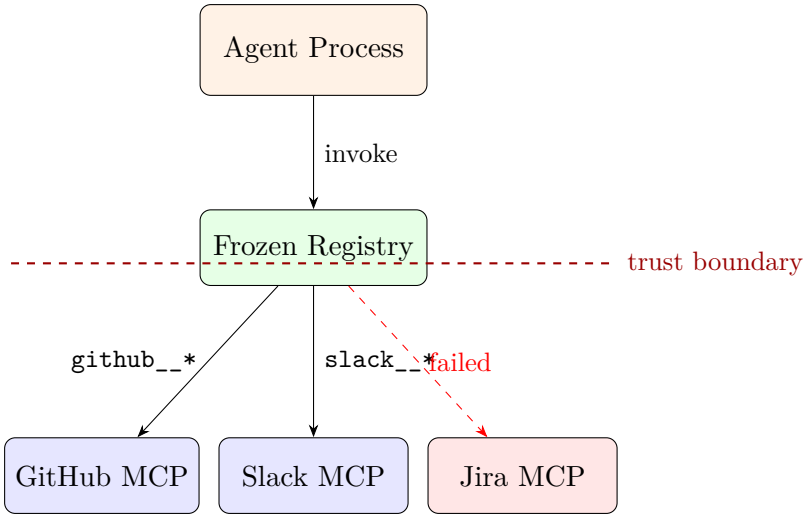

Figure 3: Multi-MCP server orchestration. The Jira connection failed, but GitHub and Slack tools are available. The registry is frozen after discovery.

12 Security Limitations and Future Directions

12.1 Known Limitations

The tool interface design has several limitations:

1. **Side channels**: Sandbox isolation prevents direct data flow between agents but does not address timing side channels or resource-usage side channels.

2. **Tool implementation trust**: We assume tool implementations are correct with respect to their schemas. A malicious tool implementation could violate type contracts.

3. **MCP server trust**: MCP-discovered tools rely on the security of external servers. A compromised server could provide tools with misleading descriptions or malicious implementations.

4. **Prompt injection**: Tools that process external content (e.g., `web-scrape`) may return data containing prompt injection attacks. The type system validates structure but not semantic content.

5. **Signing key management**: The current implementation uses a hardcoded signing key. Production deployment requires integration with a secrets manager (AWS KMS, HashiCorp Vault, etc.).

6. **Capability delegation**: The current model does not support agent-to-agent capability delegation. An agent cannot grant a subset of its capabilities to a sub-agent.

12.2 Future Directions

1. **Capability delegation**: Allowing agents to delegate subsets of their capabilities to sub-agents, with formal guarantees that delegated capabilities cannot exceed the delegator's authority.

2. **Schema-level composition checking**: Verifying at registration time that composed tool pipelines have compatible input/output schemas, catching type mismatches before execution.

3. **Formal verification**: Machine-checking the security properties using property-based testing (StreamData in Elixir) or formal verification tools (Lean 4, TLA+).

4. **Differential privacy**: Extending the audit system with differential privacy guarantees on aggregated tool usage statistics.

5. **WebAssembly sandboxing**: Adding a WASI-based sandbox tier for tools that require stronger isolation than BEAM processes provide (e.g., tools executing untrusted native code).

6. **Distributed registry**: Extending the registry to span multiple BEAM nodes using distributed Erlang, with capability tokens validated locally and tool state replicated via CRDTs.

13 Related Work

13.1 Tool Use in Language Models

The emergence of tool-augmented language models (Schick et al., 2024) has created a need for structured tool interfaces. OpenAI's function calling (OpenAI, 2024) provides JSON Schema-based tool definitions, and the Vercel AI SDK (Vercel, 2024) extends this with type-safe tool definitions using Zod schemas. The Model Context Protocol (MCP Specification, 2024) standardizes tool discovery and invocation across providers.

Our contribution is orthogonal to these interfaces: we focus on the operating system layer that manages, secures, and audits tool access, regardless of which tool invocation protocol is used at the application layer.

13.2 Capability-Based Security

The capability security literature (Dennis & Van Horn, 1966; Levy, 1984; Miller et al., 2003) provides the theoretical foundation for our token model. Capsicum (Watson et al., 2010) and seL4 (Klein et al., 2009) demonstrate that capabilities can be both practical and formally verifiable. Macaroons (Birgisson et al., 2014) extend capabilities with contextual caveats, a direction we may pursue for conditional tool access.

13.3 Actor Model and BEAM

The BEAM virtual machine realizes the actor model (Hewitt et al., 1973) with added supervision and hot-code loading (Armstrong, 2003). The tool interface's use of process isolation for sandboxing is a natural application of BEAM's architectural properties. Juric (Juric, 2019) and Thomas (Thomas, 2018) provide comprehensive treatments of Elixir/OTP patterns that inform our GenServer and supervision designs.

13.4 Operating System Abstractions

Tanenbaum and Bos (Tanenbaum & Bos, 2014) survey modern OS design. Love (Love, 2010) details the Linux kernel's device driver model. Pike et al. (Pike et al., 1995) describe Plan 9's uniform resource abstraction. Ritchie and Thompson (Ritchie & Thompson, 1978) established the foundational Unix abstractions. Our tool interface layer draws on all of these traditions, adapting them to the AI agent context.

14 Conclusion

We have presented the design and implementation of a tool interface layer for an AI Operating System. The key contributions are:

1. A **three-tier tool registry** (Builtin, Sandbox, MCP) implemented as an Elixir GenServer with freeze semantics that lock the tool set at execution time, preventing privilege escalation.

2. A **capability token system** using HMAC-SHA256 signed tokens with TTL, rate limiting, and permission scoping, providing decentralized authorization without central bottlenecks.

3. A **process-isolated sandbox executor** using BEAM `spawn_monitor` with hard timeouts, providing the same isolation guarantees as E2B ephemeral environments at the language-runtime level.

4. An **MCP protocol client** with session management, parallel multi-server discovery, and closure-based remote executors that make network boundaries transparent.

5. An **asynchronous audit logger** with telemetry integration, sensitive data sanitization, and bounded ring-buffer storage.

6. A **facade module** that composes these components into a single invocation pipeline: authorize, lookup, validate, execute, audit.

The tool interface layer is the bridge between intelligence and action. Just as Unix's device driver model enabled a rich ecosystem of applications by abstracting hardware complexity, the AI OS tool layer enables a rich ecosystem of agents by abstracting the complexity of external system interaction. The functional design ensures that tools compose cleanly, capabilities are enforced at every invocation, execution is isolated by trust level, and every action is auditable.

The Elixir/OTP reference implementation demonstrates that the BEAM virtual machine—with its per-process heap isolation, preemptive scheduling, and supervision trees—is a natural fit for implementing operating system abstractions. The approximately 700 lines of Elixir that constitute the reference implementation provide the same security and reliability guarantees that classical operating systems achieve through hardware protection rings, but with the development velocity and composability of a high-level functional language.

In Part III of this series, we turn to the memory and persistence layer—the mechanism by which agents maintain state across interactions, leveraging Elixir's ETS tables, persistent term storage, and distributed state management.

References

Armstrong, J. (2003). *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, Royal Institute of Technology, Stockholm.

Birgisson, A., Politz, J. G., Erlingsson, Ú., Taly, A., Vrable, M., and Lentczner, M. (2014). Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *NDSS 2014*.

Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. In *NeurIPS 2020*.

Cesarini, F. and Thompson, S. (2009). *Erlang Programming*. O'Reilly Media.

Dennis, J. B. and Van Horn, E. C. (1966). Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155.

E2B (2024). E2B Code Interpreter documentation. https://e2b.dev/docs.

Fong, B. and Spivak, D. I. (2019). *An Invitation to Applied Category Theory: Seven Sketches in Compositionality*. Cambridge University Press.

Hardy, N. (1988). The KeyKOS architecture. *ACM SIGOPS Operating Systems Review*, 19(4):8–25.

Hewitt, C., Bishop, P., and Steiger, R. (1973). A universal modular ACTOR formalism for artificial intelligence. In *IJCAI 1973*, pages 235–245.

Juric, S. (2019). *Elixir in Action*. Manning Publications, 2nd edition.

Klein, G., Elphinstone, K., Heiser, G., et al. (2009). seL4: Formal verification of an OS kernel. In *SOSP 2009*, pages 207–220.

Levy, H. M. (1984). *Capability-Based Computer Systems*. Digital Press.

Long, M. (2026a). The AI Operating System, Part I: Kernel architecture—process model, scheduling, and categorical foundations. *GrokRxiv preprint*, 2026.03.

Love, R. (2010). *Linux Kernel Development*. Addison-Wesley, 3rd edition.

Miller, M. S., Yee, K.-P., and Shapiro, J. (2003). Capability myths demolished. Technical report, HP Laboratories.

Model Context Protocol Working Group (2024). Model Context Protocol specification, version 1.0. https://modelcontextprotocol.io/specification.

Moggi, E. (1991). Notions of computation and monads. *Information and Computation*, 93(1):55–92.

OpenAI (2024). Function calling — OpenAI API documentation. https://platform.openai.com/docs/guides/function-calling.

Pike, R., Presotto, D., Dorward, S., Flandrena, B., Thompson, K., Trickey, H., and Winterbottom, P. (1995). Plan 9 from Bell Labs. *Computing Systems*, 8(3):221–254.

Ritchie, D. M. and Thompson, K. (1978). The UNIX time-sharing system. *Bell System Technical Journal*, 57(6):1905–1929.

Schick, T., Dwivedi-Yu, J., Dessì, R., et al. (2024). Toolformer: Language models can teach themselves to use tools. In *NeurIPS 2023*.

Shapiro, J. S. and Miller, M. S. (2023). Object-capability security: A retrospective. *IEEE Security & Privacy*, 21(2):63–71.

Tanenbaum, A. S. and Bos, H. (2014). *Modern Operating Systems*. Pearson, 4th edition.

Thomas, D. (2018). *Programming Elixir 1.6*. Pragmatic Bookshelf.

Vercel (2024). Vercel AI SDK documentation, v6. https://sdk.vercel.ai/docs.

Wadler, P. (1995). Monads for functional programming. In *Advanced Functional Programming*, LNCS 925, pages 24–52. Springer.

WASI Working Group (2023). WebAssembly System Interface specification. https://wasi.dev.

Watson, R. N. M., Anderson, J., Laurie, B., and Kennaway, K. (2010). Capsicum: Practical capabilities for UNIX. In *USENIX Security 2010*, pages 29–46.